

Inteligencia Artificial 2027-I

Tarea 2. CSP, Metaheurísticas y Búsqueda con Adversarios

Posgrado en Ciencias e Ingeniería de la Computación, UNAM

4 de septiembre de 2026

(10 puntos) Crítica de

- (5 puntos) Tönshoff, J., Kisin, B., Lindner, J., & Grohe, M. (2023). One Model, Any CSP: Graph Neural Networks as Fast Global Search Heuristics for Constraint Satisfaction. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI-23)*, pp. 4280–4288.
- (5 puntos) Doerr, B., Echarchaoui, A., Jamal, M., & Krejca, M. S. (2024, March). Runtime Analysis of the $(\mu + 1)$ GA: Provable Speed-Ups from Strong Drift towards Diverse Populations. In *Proceedings of the AAAI Conference on Artificial Intelligence* (Vol. 38, No. 18, pp. 20683–20691).

(45 puntos) Implementar en el lenguaje de su preferencia:

- (20 puntos) Búsqueda con backtracking, revisión hacia adelante, AC3 y ordenamiento
- (20 puntos) Recocido simulado o Búsqueda tabú o Algoritmo genético
- (5 puntos) Minimax

(25 puntos) Aplicar en:

- (5 puntos) N-reinas con $N = 8$ y 100 (resolver con backtracking y la metaheurística)
- (5 puntos) Encontrar todas las soluciones para N-reinas con $N = 100$ (con el método de su preferencia)
- (5 puntos) Problema de coloreado de grafos (50 y 1000 nodos) (resolver con backtracking y la metaheurística)
- (5 puntos) Resolver el mismo problema de coloreado de grafos (50 y 1000 nodos) con OR-Tools, y comparar tiempo de cómputo y calidad de la solución (número de colores) contra los resultados de backtracking y de la metaheurística
- (5 puntos) Juego de gato de <https://github.com/LudwigStumpp/gym-tic-tac-toe> (resolver con minimax)

(20 puntos) Ejercicios

Ejercicio 1. (5 puntos) Considere el problema de colocar k caballos en un tablero de ajedrez de $n \times n$ de forma que ningún par de caballos se ataque entre sí, donde k está dado y $k \leq n^2$.

- (a) Elija una formulación como CSP. En su formulación, ¿cuáles son las variables?

- (b) ¿Cuáles son los valores posibles de cada variable?
- (c) ¿Qué conjuntos de variables están restringidos, y cómo?
- (d) Considere ahora el problema de colocar la mayor cantidad posible de caballos en el tablero sin que se ataquen entre sí. Explique cómo resolver esto con búsqueda local, definiendo las funciones ACCIONES y RESULTADO apropiadas y una función objetivo razonable.

Ejercicio 2. (5 puntos) Presentamos Sudoku como un CSP que se resuelve mediante búsqueda sobre asignaciones parciales, porque así es como la gente suele resolver Sudokus. También es posible, desde luego, atacar estos problemas con búsqueda local sobre asignaciones completas. ¿Qué tan bien le iría a un solucionador local que use una heurística que minimize el número de restricciones violadas en problemas de Sudoku?

Ejercicio 3. (5 puntos) Este problema ejercita los conceptos básicos de la búsqueda con adversarios usando el gato (tic-tac-toe) como ejemplo. Defina X_n como el número de filas, columnas o diagonales con exactamente n X's y ninguna O. De forma análoga, O_n es el número de filas, columnas o diagonales con exactamente n O's. La función de utilidad asigna +1 a cualquier posición con $X_3 = 1$ y -1 a cualquier posición con $O_3 = 1$. El resto de las posiciones terminales tienen utilidad 0. Para las posiciones no terminales use la función de evaluación lineal $\text{Eval}(s) = 3X_2(s) + X_1(s) - (3O_2(s) + O_1(s))$.

- (a) ¿Aproximadamente cuántas partidas posibles de gato existen?
- (b) Muestre el árbol de juego completo desde el tablero vacío hasta profundidad 2 (es decir, con una X y una O en el tablero), tomando en cuenta la simetría.
- (c) Marque en su árbol las evaluaciones de todas las posiciones en profundidad 2.
- (d) Usando el algoritmo minimax, marque en su árbol los valores propagados hacia las posiciones en profundidad 1 y 0, y úselos para elegir la mejor jugada inicial.
- (e) Marque con un círculo los nodos en profundidad 2 que no se evaluarían si se aplicara poda alfa-beta, suponiendo que los nodos se generan en el orden óptimo para dicha poda.

Ejercicio 4. (5 puntos) Desarrolle una prueba formal de correctud para la poda alfa-beta. Considere la situación en la que un nodo n_j , que es un nodo max, es descendiente de un nodo n_1 . La idea básica es podar n_j si y solo si se puede mostrar que el valor minimax de n_1 es independiente del valor de n_j .

- (a) El nodo n_1 toma el valor mínimo entre sus hijos, $n_1 = \min(n_2, n_{21}, \dots, n_{2b_2})$. Encuentre una expresión análoga para n_2 y, a partir de ella, una expresión para n_1 en términos de n_j .
- (b) Sea l_i el valor mínimo (o máximo) de los nodos a la izquierda del nodo n_i en profundidad i , cuyo valor minimax ya se conoce. De forma análoga, sea r_i el valor mínimo (o máximo) de los nodos aún no explorados a la derecha de n_i en profundidad i . Reescriba su expresión para n_1 en términos de los valores l_i y r_i .

- (c) Reformule la expresión para mostrar que, para afectar el valor de n_1 , n_j no debe exceder una cierta cota derivada de los valores l_i .
- (d) Repita el proceso para el caso en que n_j es un nodo min.

Notas, puntos negativos en caso de no entregarlo:

- Se debe entregar un **archivo pdf** de reporte de la tarea.
- El reporte debe explícitamente mencionar cuáles partes de la tarea se resolvieron.
- El reporte debe contener las críticas a las lecturas, las soluciones encontradas a los diferentes problemas y las funciones objetivo definidas para cada caso.
- Se debe entregar el **código funcional** para reproducir los resultados así como las instrucciones para su ejecución.
- Para el coloreado de grafos, la primera línea corresponde al número de vértices y aristas. Las líneas subsecuentes corresponden a las aristas del problema.
- La salida del problema de coloreado de grafos debe ser: en la primera línea el número de colores utilizado y en las líneas subsecuentes el color utilizado seguido del vértice.